

Software Requirements Specification

Tiered-Permission CLI AI Agent

(Working name: “gatekeeper-cli”)

Version 0.1 — Draft

Author: Shreyas

Date: August 4, 2026

1. Introduction

1.1 Purpose

This document specifies the requirements for a command-line AI agent that executes tasks on a user's machine — via direct shell commands and via MCP (Model Context Protocol) tool calls — under a tiered permission-gating system. The gate intercepts every proposed action, classifies its risk, and requires explicit user confirmation with a plain-language explanation before executing anything above a defined risk threshold.

The system does not attempt to alter or bypass the underlying LLM's own safety behavior. It is an operational safety layer around command execution, independent of and complementary to the model's judgment.

1.2 Scope

The product, referred to as the Agent, will:

- Accept natural-language tasks from a user via a CLI.
- Use an LLM to plan and propose actions — shell commands and/or MCP tool calls.
- Classify every proposed action against a five-tier risk ruleset before execution.
- For tiers requiring confirmation, generate a plain-language explanation of what the action does and why it was flagged, then block on user approval.
- Execute approved actions in a scoped/sandboxed environment and log all proposals, approvals, denials, and outcomes.
- Support pluggable MCP servers as tool providers, each independently scoped and each tool independently tiered.

Out of scope for v1: multi-user/team permission delegation, remote/headless approval (e.g. mobile push approval), and fine-grained per-file ACLs beyond directory scoping.

1.3 Definitions, Acronyms, Abbreviations

Term	Definition
Agent	The CLI application described by this SRS.
MCP	Model Context Protocol — standard for exposing tools/resources to an LLM via a client–server interface.
MCP Host / Client	The orchestrator process that dispatches tool calls to MCP servers; the Agent implements this role.
Tool call	A structured, typed invocation of a capability (shell exec, file write, HTTP request, etc.), whether native or via MCP.
Tier	One of five risk levels (T1–T5) assigned to a tool or command pattern; see §3.2.
Gate	The interception layer that classifies a proposed action and enforces the tier policy before execution.
Ruleset	The versioned configuration mapping command patterns / MCP tool names to tiers.
Audit log	Append-only record of every proposed action, its tier, the decision, and the outcome.

1.4 References

- Model Context Protocol specification (Anthropic)
- OWASP command-injection and least-privilege guidelines
- IEEE 830 / ISO/IEC/IEEE 29148 (SRS structure this document loosely follows)

2. Overall Description

2.1 Product Perspective

The Agent is a standalone CLI tool, not a hosted service. It runs locally, holds no server-side state, and talks to:

- An LLM API (model-agnostic; Claude, GPT-class, or local via Ollama/DeepInfra) for planning and explanation generation.
- Zero or more MCP servers (local or remote) for tool execution (filesystem, git, shell, browser, DB, etc.).
- The local OS shell, sandboxed via container/VM where configured, for direct command execution.

The Agent itself is the MCP Host: it owns the tool-call dispatch loop and is the single point where the permission gate is enforced. Individual MCP servers are not trusted to self-report risk.

2.2 Product Functions (Summary)

- Natural-language task intake and LLM-driven planning.
- Command/tool-call risk classification (static, deterministic — not LLM self-assessment).
- Tiered confirmation flow with LLM-generated, human-readable explanations.
- Sandboxed execution and structured audit logging.
- Ruleset management (view, edit, test tiers without executing).
- MCP server registration, per-server scoping, and per-tool tier overrides.

2.3 User Characteristics

Primary user: a single technical operator (developer/sysadmin) running the Agent on their own workstation or a scoped server, comfortable reading shell commands and making a judgment call when prompted. Not designed for non-technical end users in v1 — explanations assume basic CLI literacy.

2.4 Operating Environment

- OS: Linux and WSL2 primary target; macOS secondary. Windows-native not required for v1.
- Runtime: Python 3.11+ (reference implementation) or Go (alternative for single-binary distribution).
- Optional container runtime (Docker) for sandboxed execution mode.
- Network egress required for LLM API calls and remote MCP servers; local-only mode possible with a local model.

2.5 Design and Implementation Constraints

- The classifier must be deterministic and independent of the LLM — the LLM proposes actions and writes explanations, but never decides whether the gate opens.
- Command parsing must use a real shell grammar parser (e.g. bashlex), not substring/regex matching alone, to resist trivial obfuscation.
- Default-deny for unrecognized MCP tools: anything without an explicit ruleset entry is treated as Tier 4 minimum, never auto-approved.
- All state (ruleset, audit log) stored locally in plain, human-readable formats (YAML/JSONL) — no opaque binary state.

2.6 Assumptions and Dependencies

- User has valid API credentials for at least one LLM provider.
- MCP servers used are reasonably well-behaved (correct tool schemas); the Agent does not attempt to sandbox against a malicious MCP server's protocol violations in v1.
- Sandboxing (containers) is assumed available but is not a hard requirement — the confirmation gate must function standalone.

3. System Features / Functional Requirements

3.1 Task Intake and Planning (FR-1.x)

ID	Requirement
FR-1.1	The Agent shall accept a natural-language task via CLI prompt or piped stdin.
FR-1.2	The Agent shall send the task plus relevant context (cwd, recent history, available tools/MCP servers) to the configured LLM for planning.
FR-1.3	The LLM's plan shall be decomposed into a sequence of discrete, individually-classifiable actions (shell commands or MCP tool calls) — not executed as an opaque multi-step blob.
FR-1.4	The Agent shall support interactive (step-by-step) and batch (plan-then-review) execution modes, user-selectable.

3.2 Risk Classification / Tier System (FR-2.x)

Every proposed action is assigned exactly one tier by the deterministic classifier before execution:

Tier	Label	Examples	Gate behavior
T1	Read-only	ls, cat, git status, list MCP resources	Auto-approve, log only
T2	Safe write	write to scratch/temp dir, git diff, create file in project scope	Auto-approve, log with detail
T3	Moderate risk	package install (scoped venv), git push, edit tracked file, external API GET	Confirm + explanation required
T4	Destructive / hard-to-reverse	rm -rf, chmod -R, DB DROP/TRUNCATE, force-push, kill process	Confirm + explanation + typed confirmation phrase
T5	Irreversible / high blast radius	sudo, credential/secret access, network calls shaped like exfil, system-level config (iptables, systemctl), prod DB writes	Deny by default; requires explicit CLI override flag at launch, then still confirm + typed phrase

3.2.1 Classification requirements

ID	Requirement
FR-2.1	Classification shall be performed by a deterministic ruleset engine, never by asking the LLM to self-rate its own proposed action.
FR-2.2	Shell commands shall be parsed via a shell-grammar parser (e.g. bashlex) to correctly classify piped, chained, and variable-obfuscated commands, not via naive substring match.
FR-2.3	MCP tool calls shall be classified by (server, tool name) tuple against the ruleset; unmapped tools default to Tier 4.
FR-2.4	The ruleset shall be a versioned, human-editable config file (YAML) supporting pattern rules (regex/glob) for shell and exact/wildcard rules for MCP tool names.
FR-2.5	The user shall be able to add, override, or tighten (but not silently loosen below T3) tier assignments via a ruleset editing command.

3.3 Explanation and Confirmation Flow (FR-3.x)

ID	Requirement
FR-3.1	For any action classified T3–T5, the Agent shall generate a plain-language explanation covering: what the command/tool does, its scope of effect, and why it was flagged (which rule matched).
FR-3.2	The explanation shall be generated by the LLM but shall not be permitted to alter the tier or bypass the confirmation step — explanation generation and gate enforcement are separate code paths.
FR-3.3	T3 actions require a simple y/n confirmation.
FR-3.4	T4 actions require the user to type a specific confirmation phrase (not just y/n) to reduce accidental approval.
FR-3.5	T5 actions are denied unless the Agent was launched with an explicit override flag (e.g. --allow-tier5); even then, T4-style typed confirmation is still required per action.
FR-3.6	On denial, the Agent shall report the denial to the LLM planning loop so it can propose an alternative rather than silently retrying the same action.

3.4 Execution and Sandboxing (FR-4.x)

ID	Requirement
FR-4.1	Approved actions shall execute via a subprocess wrapper capturing stdout, stderr, exit code, and wall-clock duration.
FR-4.2	The Agent shall support an optional sandboxed execution mode (container/VM) applied regardless of tier, as a second line of defense independent of classifier correctness.
FR-4.3	Execution timeouts shall be configurable per tier (lower default timeout for higher tiers).
FR-4.4	Failed actions shall surface the error to the LLM planning loop with enough detail to allow re-planning.

3.5 Audit Logging (FR-5.x)

ID	Requirement
FR-5.1	Every proposed action shall be logged regardless of tier or outcome: timestamp, action detail, tier, matched rule, decision (auto/approved/denied), and result.
FR-5.2	The audit log shall be append-only (JSONL), locally stored, and never silently truncated or rotated without explicit user action.
FR-5.3	The Agent shall provide a query/report command to view logs filtered by tier, date range, or outcome.

3.6 MCP Integration (FR-6.x)

ID	Requirement
FR-6.1	The Agent shall act as an MCP Host, supporting registration of multiple MCP servers (local stdio or remote URL/SSE).
FR-6.2	Each MCP server shall be independently scoped (e.g. filesystem server restricted to a declared root directory).
FR-6.3	Tool schemas exposed by MCP servers shall be mapped into the ruleset at registration time; tools without an explicit mapping default to Tier 4 (see FR-2.3).
FR-6.4	The gate shall apply identically to native shell actions and MCP tool calls — no MCP tool bypasses classification.

3.7 Ruleset Management (FR-7.x)

ID	Requirement
FR-7.1	The Agent shall provide a CLI subcommand to list current tier mappings.
FR-7.2	The Agent shall provide a dry-run mode: classify a given command/tool call and show its tier and explanation without executing.
FR-7.3	Ruleset changes shall themselves be logged (who/when/what changed) in the audit log.

4. Non-Functional Requirements

4.1 Security

- The gate must fail closed: any classifier error, parse failure, or ambiguous command shall default to the higher of the two candidate tiers, never the lower.
- Secrets (API keys, tokens) shall never be written to the audit log in plaintext; redact known secret patterns before logging.
- The Agent shall not execute LLM-proposed actions that were not routed through the classifier, under any code path, including error-handling and retry logic.

4.2 Reliability

- Audit log writes shall be synchronous and durable (fsync) before an action is reported as executed, so a crash mid-action leaves an accurate record.

4.3 Usability

- Explanations shall be understandable to a technical user without requiring them to read the raw command — plain language first, raw command shown alongside for verification.
- Confirmation prompts shall clearly display the tier and the specific rule that triggered the gate.

4.4 Performance

- Classification of a single proposed action shall complete in under 50ms (excluding LLM explanation generation) so it does not become the bottleneck relative to LLM latency.

4.5 Portability

- The ruleset format and audit log format shall be plain text (YAML/JSONL) to remain portable across the Python and any future Go implementation.

5. Open Questions / Future Considerations

1. Should Tier 5 override support a scoped, time-limited elevation (e.g. “allow tier 5 for next 10 minutes”) rather than a whole-session flag?
2. Should the ruleset support per-directory or per-project overrides (e.g. looser rules inside a disposable container, stricter on the host)?
3. Multi-user / team approval delegation is out of scope for v1 — revisit if the tool moves beyond single-operator use.
4. Evaluate whether MCP server-declared tool “annotations” (if/when standardized) can inform — but not replace — the local tier mapping.